

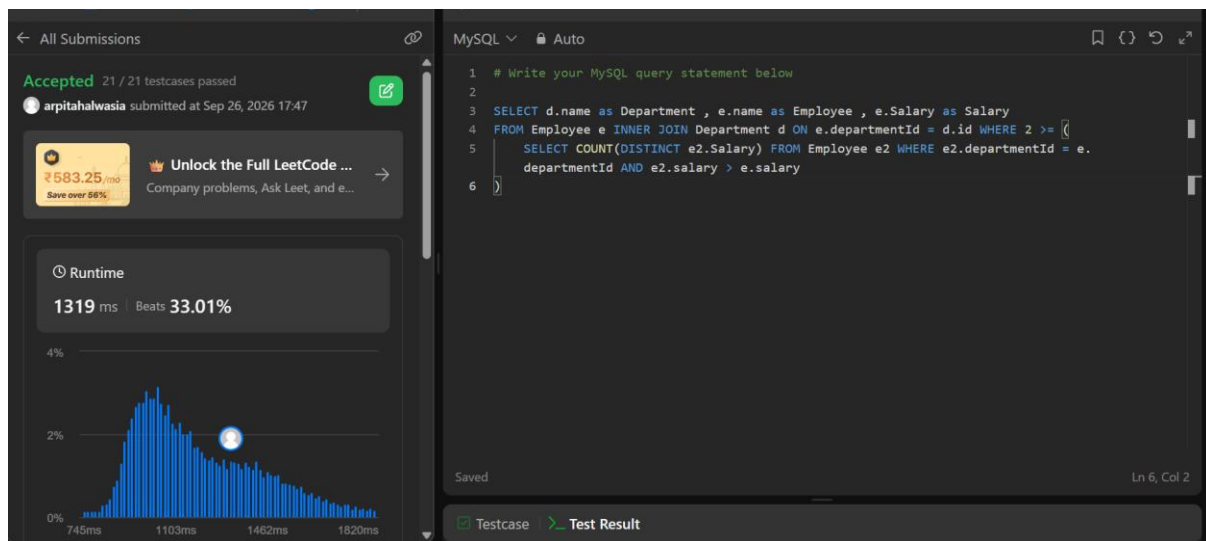
ASSIGNMENT – 4

Question – 1 (LeetCode 185 – Department Top Three Salaries)

Code –

```
SELECT d.name as Department , e.name as Employee , e.Salary as Salary
FROM Employee e INNER JOIN Department d ON e.departmentId = d.id WHERE 2 >= (
    SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2.departmentId =
e.departmentId AND e2.salary > e.salary
)
```

Output –



Question – 2(LeetCode 262 – Trips and Users)

Code –

```
SELECT t.request_at AS Day ,
ROUND(
    SUM(
        CASE
            WHEN t.status IN ('cancelled_by_driver' , 'cancelled_by_client')
            THEN 1
```

ELSE 0

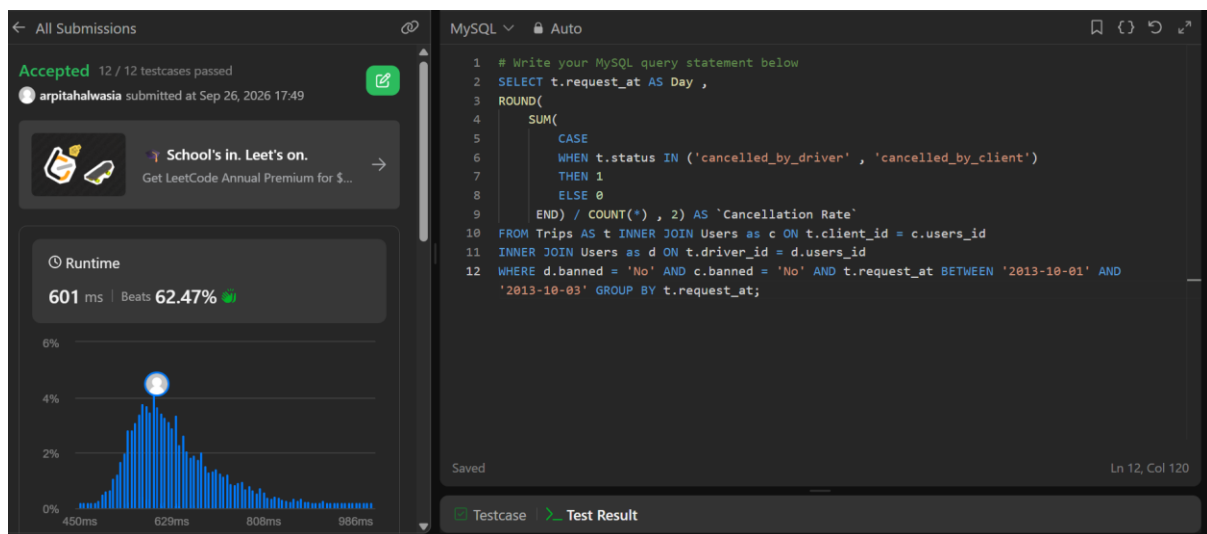
END) / COUNT(*) , 2) AS `Cancellation Rate`

FROM Trips AS t INNER JOIN Users as c ON t.client_id = c.users_id

INNER JOIN Users as d ON t.driver_id = d.users_id

WHERE d.banned = 'No' AND c.banned = 'No' AND t.request_at BETWEEN '2013-10-01'
AND '2013-10-03' GROUP BY t.request_at;

Output –



Question – 3(LeetCode 601 – Human Traffic of Stadium)

Code –

WITH HIGH_TRAFFIC AS (

SELECT * , ROW_NUMBER() OVER (ORDER BY id) AS rn

FROM Stadium WHERE

people >= 100

),

group_data AS (

SELECT * , id-rn AS grp FROM HIGH_TRAFFIC

)

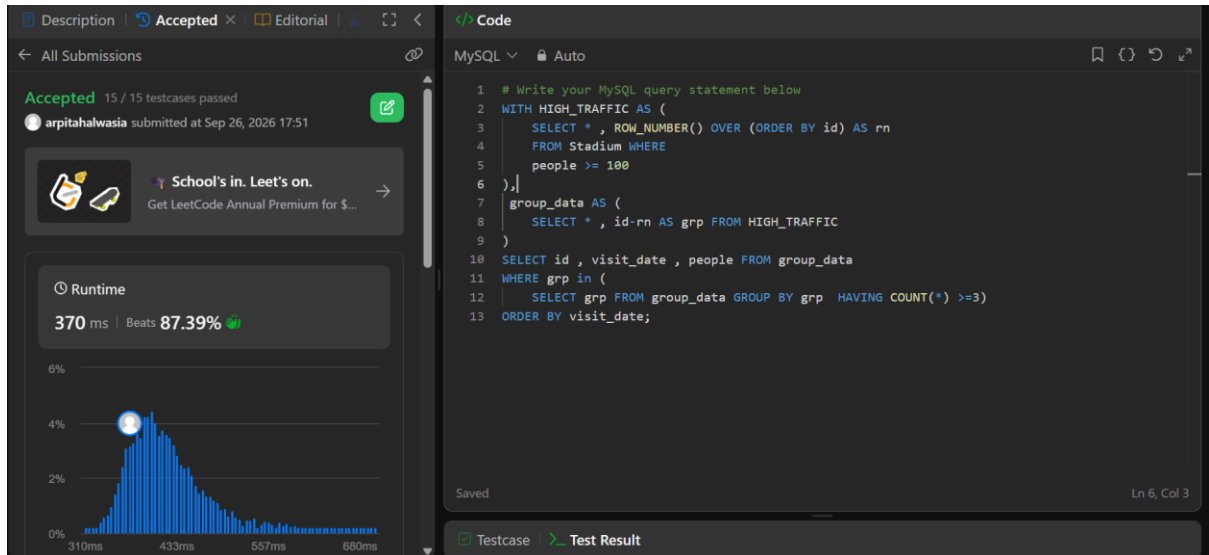
SELECT id , visit_date , people FROM group_data

WHERE grp in (

SELECT grp FROM group_data GROUP BY grp HAVING COUNT(*) >=3)

ORDER BY visit_date;

Output –



Question – 4 (LeetCode 3374 – First Letter Capitalization II)

Code –

WITH RECURSIVE converted AS (

SELECT

content_id,

content_text,

1 AS pos,

CAST(' AS CHAR(10000)) AS result,

1 AS word_start,

1 AS valid_word,

0 AS hyphenated_word

FROM user_content

UNION ALL

SELECT

content_id,

content_text,

pos + 1,

CONCAT(

result,

CASE

WHEN SUBSTRING(content_text, pos, 1) = ' ' THEN ' '

WHEN word_start = 1

AND SUBSTRING(content_text, pos, 1) NOT REGEXP '[A-Za-z]'

THEN SUBSTRING(content_text, pos, 1)

WHEN valid_word = 0

THEN SUBSTRING(content_text, pos, 1)

WHEN word_start = 1

THEN UPPER(SUBSTRING(content_text, pos, 1))

WHEN hyphenated_word = 1

AND SUBSTRING(content_text, pos - 1, 1) = '-'

AND SUBSTRING(content_text, pos, 1) REGEXP '[A-Za-z]'

THEN UPPER(SUBSTRING(content_text, pos, 1))

WHEN SUBSTRING(content_text, pos, 1) REGEXP '[A-Za-z]'

THEN LOWER(SUBSTRING(content_text, pos, 1))

```

        ELSE
            SUBSTRING(content_text, pos, 1)
        END
    ),

CASE
    WHEN SUBSTRING(content_text, pos, 1) = ' ' THEN 1
    ELSE 0
END,

CASE
    WHEN SUBSTRING(content_text, pos, 1) = ' ' THEN 1

    WHEN word_start = 1
        AND SUBSTRING(content_text, pos, 1) NOT REGEXP '^[A-Za-z]$\
        THEN 0

    ELSE valid_word
END,

CASE
    WHEN SUBSTRING(content_text, pos, 1) = ' ' THEN 0

    WHEN word_start = 1
        AND SUBSTRING(content_text, pos, 1) REGEXP '^[A-Za-z]$\
        THEN
            CASE
                WHEN SUBSTRING(

```

```

        content_text,
        pos,
        LOCATE(
            '',
            CONCAT(SUBSTRING(content_text, pos), ' ')
        ) - 1
    ) REGEXP '^[A-Za-z]+(-[A-Za-z]+)+$'
    THEN 1
    ELSE 0
END

ELSE hyphenated_word

END

FROM converted

WHERE pos <= CHAR_LENGTH(content_text)
)

SELECT
    content_id,
    content_text AS original_text,
    result AS converted_text
FROM converted
WHERE pos = CHAR_LENGTH(content_text) + 1
ORDER BY content_id;

```

Output –

Code

```

1 WITH RECURSIVE converted AS (
2     SELECT
3         content_id,
4         content_text,
5         1 AS pos,
6         CAST('' AS CHAR(1000)) AS result,
7

```

Runtime
556 ms | Beats 9.59%

Testcase | **Test Result**
Accepted Runtime: 74 ms

Case 1

Input

```

user_content =
| content_id | content_text |
|-----|-----|
| 1 | hello world of SQL |
| 2 | the QUICK-brown fox |

```

Question – 5(LeetCode 3451 – Find Invalid IP Addresses)

Code –

SELECT

ip,

COUNT(*) AS invalid_count

FROM logs

WHERE ip NOT REGEXP

'^((0|[1-9][0-9]?|1[0-9]{2}|2[0-4][0-9]|25[0-5])\\.){3}(0|[1-9][0-9]?|1[0-9]{2}|2[0-4][0-9]|25[0-5])\$'

GROUP BY ip

ORDER BY invalid_count DESC, ip DESC;

Output –

Code

```

8 [0-9]|25[0-5])$'
9 GROUP BY ip
10 ORDER BY invalid_count DESC, ip DESC;

```

Runtime
276 ms | Beats 72.36%

Testcase | **Test Result**
Accepted Runtime: 95 ms

Case 1

Input

```

logs =
| log_id | ip | status_code |
|-----|-----|-----|
| 8 | 127.0.0.1 | 200 |
| 9 | 192.168.1 | 400 |

```

Question – 6(LeetCode 3482 – Analyze Organization Hierarchy)

Code –

WITH RECURSIVE

levels AS (

SELECT

employee_id,employee_name,manager_id,salary,department, 1 AS level FROM
Employees

WHERE manager_id IS NULL

UNION ALL

SELECT

e.employee_id,e.employee_name,e.manager_id,e.salary,e.department,l.level + 1
FROM Employees e

JOIN levels l

ON e.manager_id = l.employee_id

),

hierarchy AS (

SELECT employee_id AS ancestor,employee_id AS employee FROM Employees

UNION ALL

SELECT h.ancestor,e.employee_id

FROM hierarchy h

JOIN Employees e

ON e.manager_id = h.employee

)

SELECT

**l.employee_id,l.employee_name, l.level, COUNT(h.employee) - 1 AS team_size,
SUM(e.salary) AS budget**

FROM levels l

JOIN hierarchy h

ON l.employee_id = h.ancestor

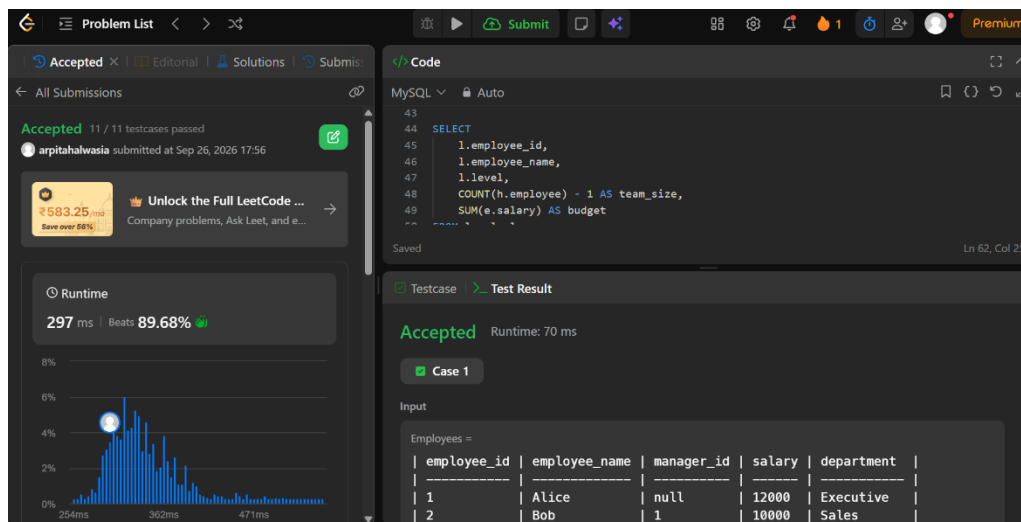
JOIN Employees e

ON h.employee = e.employee_id

GROUP BY l.employee_id, l.employee_name, l.level

ORDER BY l.level ASC,budget DESC, l.employee_name ASC;

Output –



Question – 7(LeetCode 3554 – Find Category Recommendation Pairs)

Code –

WITH user_categories AS (

SELECT DISTINCT p.user_id, i.category

FROM ProductPurchases p

JOIN ProductInfo i

ON p.product_id = i.product_id

)

SELECT

a.category AS category1,

b.category AS category2,

COUNT(DISTINCT a.user_id) AS customer_count

FROM user_categories a

JOIN user_categories b

ON a.user_id = b.user_id

AND a.category < b.category

GROUP BY a.category, b.category

HAVING COUNT(DISTINCT a.user_id) >= 3

ORDER BY

customer_count DESC,

category1 ASC,

category2 ASC;

Output –

The screenshot displays the LeetCode submission interface for a MySQL problem. The top section shows the submission status as "Accepted" with 12 out of 12 testcases passed. The user "arpitahawasia" submitted the solution at Sep 26, 2026 18:00. Below this, there is a promotional banner for "School's in. Leet's on." and a "Runtime" section indicating a time of 575 ms, which beats 98.45% of other submissions. A bar chart shows the distribution of runtime times, with the user's submission marked by a blue circle. The bottom right section shows the MySQL query code in a dark-themed editor, which includes a CTE named "user_categories" and a final SELECT statement. The test result section below the code is empty, displaying the message "You must run your code first".

MySQL Auto

```
1 # Write your MySQL query statement below
2 WITH user_categories AS (
3     SELECT DISTINCT p.user_id, i.category
4     FROM ProductPurchases p
5     JOIN ProductInfo i
6     ON p.product_id = i.product_id
```

Saved Ln 16, Col 28

Testcase Test Result

You must run your code first